EXPT NO: 7	A PYTHON PROGRAM TO SOLVE DYNAMIC PROGRAMMING PROBLEMS
DATE:	

AIM:

• To maximize the total value of items included in a knapsack without exceeding its capacity, given weights and values of the items

• To determine if a subset of a given set of integers sums up to a specified value.

• To find the length of the longest subsequence common to two sequences, where the subsequence maintains the order of elements.

PROCEDURE:

1. KNAPSACK PROBLEM:

• To maximize the total value of items included in a knapsack without exceeding its capacity, given weights and values of the items

2. SUBSET SUM PROBLEM:

• To Determine Whether There Exists A Subset Of The Given Set Of Numbers That

• Adds Up To A Specific Target Sum.

3. LONGEST COMMON SUBSEQUENCE

• To find the length of the longest subsequence common to the two sequences while maintaining the order of elements.

PROBLEM – 1: SOLVE THE PROBLEM USING SUBSET SUM PROBLEM.

Weights	2	3	4	5	6	7	8
Values	3	4	8	8	10	13	15
Capacity W	15						

CODE

```
def knapsack(weights, values, W):
    n = len(weights)
```

```
    dp = [[0 for _ in range(W + 1)] for _ in range(n + 1)]
    for i in range(n + 1):
        for w in range(W + 1):
            if i == 0 or w == 0:
                dp[i][w] = 0
            elif weights[i - 1] <= w:
                dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]],
                               dp[i - 1][w])
            else:
                dp[i][w] = dp[i - 1][w]
    return dp[n][W]
weights = [2, 3, 4, 5, 6, 7, 8]
values = [3, 4, 8, 8, 10, 13, 15]
W = 15
max_value = knapsack(weights, values, W)
print("Maximum value in Knapsack:", max_value)
```

OUTPUT

Maximum value in Knapsack: 28

PROBLEM – 2: SOLVE THE PROBLEM USING SUBSET SUM PROBLEM.

A knapsack has capacity **W = 10**.
Given weights and values of items:

- Weights = [1, 3, 4, 6]
- Values = [2, 5, 7, 10]

CODE

```
def knapsack(weights, values, W):
    n = len(weights)
    dp = [[0 for _ in range(W + 1)] for _ in range(n + 1)]
  for i in range(n + 1):
```

```
        for w in range(W + 1):
            if i == 0 or w == 0:
                dp[i][w] = 0
            elif weights[i - 1] <= w:
                dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]],
                               dp[i - 1][w])
            else:
                dp[i][w] = dp[i - 1][w]
    return dp[n][W]
weights = [1, 3, 4, 6]
values = [2, 5, 7, 10]
W = 10
max_value = knapsack(weights, values, W)
print("Maximum value in Knapsack:", max_value)
```

OUTPUT

Maximum value in Knapsack: 17

RESULT

Thus, Solving Knapsack Problem, Subset Sum Problem And Longest Common Subsequence Problem Using Dynamic Programming In Python Has Been Implemented And Executed Successfully.